MODULE

25

Django + DRF

The batteries-included framework most Python web jobs ask for: ORM, admin, auth and REST APIs.

YOU WILL LEARN

Guided tour	`01_django_tour.py`
The bookstore project	`bookstore_project/`
REST API with DRF	`catalog/api.py`
Tests	`catalog/tests.py`
Interview Q&A	`INTERVIEW.md`

WHY DJANGO

Why companies pick Django

THE PROBLEM

A startup needs a shop with users, an admin panel for staff, a database with migrations, forms, and an API for the mobile app. **By Friday.**

THE IDEA

Django ships all of it: ORM, migrations, admin, auth, forms, templates. DRF adds REST APIs. You write the business logic.

PICTURE IT REQUEST FLOW

How Django answers a request (MVT)

🏛️ Django calls it **Model-View-Template**: the URL router picks a view, the view uses models to talk to the DB, and a template (or a DRF serializer) turns the result into HTML or JSON. Middleware wraps everything.



STEP BY STEP PROJECT LAYOUT

A Django project at a glance

FILE	JOB
`manage.py`	runserver, migrate, makemigrations, createsuperuser, test
`config/settings.py`	INSTALLED_APPS, DATABASES, MIDDLEWARE (read secrets from env!)
`catalog/models.py`	tables as classes
`catalog/views.py · urls.py`	pages and their URLs
`catalog/admin.py`	a full admin UI in a few lines
`catalog/api.py`	DRF serializers + viewsets
`catalog/migrations/`	generated schema changes, committed to git

TRY IT ORM

QuerySets are lazy and chainable

```python
class Author(models.Model):
    name = models.CharField(max_length=100)
    class Meta: app_label = "shop"

class Book(models.Model):
    title = models.CharField(max_length=200)
    price = models.DecimalField(max_digits=6, decimal_places=2)
    author = models.ForeignKey(Author, on_delete=models.CASCADE, related_name="books")
    class Meta: app_label = "shop"

with connection.schema_editor() as e:
    e.create_model(Author); e.create_model(Book)
a = Author.objects.create(name="Ramalho")
Book.objects.create(title="Fluent Python", price=49, author=a)
Book.objects.create(title="Python Tricks", price=29, author=a)

qs = Book.objects.filter(price__lt=40).order_by("title")   # no SQL yet
print(qs.query)
print(list(qs.values_list("title", flat=True)))           # runs now
```

OUTPUT

```
SELECT "shop_book"."id",
"shop_book"."title",
"shop_book"."price",
"shop_book"."author_id" FROM
"shop_book" WHERE "shop_book"."price"
< 40 ORDER BY "shop_book"."title" ASC
['Python Tricks']
```

1. **filter / exclude / order_by** build the query; nothing runs yet.
2. It runs when you **iterate**, slice or call list().
3. **price__lt**: field lookups with double underscores.

TRY IT ORM

select_related vs prefetch_related

```python
class Author(models.Model):
    name = models.CharField(max_length=100)
    class Meta: app_label = "shop"
class Book(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey(Author, on_delete=models.CASCADE, related_name="books")
    class Meta: app_label = "shop"
with connection.schema_editor() as e:
    e.create_model(Author); e.create_model(Book)
for i in range(20):
    Book.objects.create(title=f"b{i}", author=Author.objects.create(name=f"a{i}"))

def count(fn):
    with CaptureQueriesContext(connection) as ctx:
        fn()
    return len(ctx.captured_queries)

print("lazy FK       :", count(lambda: [b.author.name for b in Book.objects.all()]))
print("select_related:", count(lambda: [b.author.name for b in Book.objects.select_related("author")]))
print("prefetch      :", count(lambda: [list(a.books.all()) for a in Author.objects.prefetch_related("books")]))
```

```
OUTPUT
lazy FK       : 21
select_related: 1
prefetch      : 2
```

1. **select_related**: FK / one-to-one, one **JOIN**.
2. **prefetch_related**: reverse FK / many-to-many, a second **IN** query.
3. Asked in almost every Django interview.

TRY IT DRF

A REST API in three pieces

```python
from rest_framework import permissions, routers, serializers, viewsets

class BookSerializer(serializers.ModelSerializer):
    class Meta:
        model = Book
        fields = ["id", "title", "price", "author"]

    def validate_price(self, value):
        if value <= 0:
            raise serializers.ValidationError("price must be positive")
        return value

class BookViewSet(viewsets.ModelViewSet):        # list/create/retrieve/update/destroy
    queryset = Book.objects.select_related("author")
    serializer_class = BookSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly]

router = routers.DefaultRouter()
router.register("books", BookViewSet)            # /books/ and /books/{id}/
```

OUTPUT

```
GET    /api/books/      -> 200 list
POST   /api/books/      -> 201
(logged in)
GET    /api/books/1/    -> 200
PATCH  /api/books/1/    -> 200
DELETE /api/books/1/    -> 204
POST price=0            -> 400
{"price": ["price must be positive"]}
```

1. **Serializer** = validation + JSON (like Pydantic).
2. **ViewSet** + **router** = all CRUD endpoints.
3. The project's tests in **catalog/tests.py** check these.

COMMON MISTAKE SETTINGS

Shipping the development settings

✗ WRONG

```
DEBUG = True
SECRET_KEY = "django-insecure-abc123"
ALLOWED_HOSTS = ["*"]
```

DEBUG pages leak code and settings to attackers; a committed SECRET_KEY lets them forge sessions.

✓ RIGHT

```
DEBUG = os.environ.get("DJANGO_DEBUG") == "1"
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
ALLOWED_HOSTS = os.environ["ALLOWED_HOSTS"].split(",")
# python manage.py check --deploy
```

Settings from the **environment**; run **manage.py check --deploy** before going live.

STEP BY STEP DJANGO FEATURES

More things interviewers ask about

FEATURE	WHAT IT IS	USE IT FOR
Middleware	code around every request/response	auth, CSRF, CORS, request IDs, timing
Signals	hooks like `post_save`	small side effects; prefer explicit service calls
Migrations	`makemigrations` + `migrate`	schema changes in git
Admin	auto CRUD UI from models	internal staff tools
transaction.atomic	all-or-nothing block	orders + stock updates
F() / Q()	DB-side math / OR conditions	`F("stock") - 1` avoids lost updates

IN REAL JOBS DJANGO

Django at work

Admin for staff in 5 lines

```
@admin.register(Book)
class BookAdmin(admin.ModelAdmin):
    list_display = ["title", "price"]
```

Atomic stock update

```
Book.objects.filter(pk=pk, stock__gt=0)
            .update(stock=F("stock") - 1)
```

Tests

```
class BookAPITests(APITestCase):
    def test_list(self): ...
```

Production

```
gunicorn config.wsgi -w 4
collectstatic · whitenoise · Postgres
```

INTERVIEW SAY THIS IN THE INTERVIEW

“select_related vs prefetch_related?”

- Both fix the **N+1** problem when you access related objects in a loop.
- **select_related** follows **ForeignKey / OneToOne** with a SQL **JOIN**: one query.
- **prefetch_related** handles **reverse FK and ManyToMany**: a second query with **IN (...)**, joined in Python.
- Verify with django-debug-toolbar or **assertNumQueries** in tests.

```
Book.objects.select_related("author")
Author.objects.prefetch_related("books")
self.assertNumQueries(2, lambda: ...)
```

Follow-up they ask next: ”Are QuerySets lazy?” (yes: they hit the DB only when evaluated: iteration, list(), len(), slicing with a step, bool()).

INTERVIEW SAY THIS IN THE INTERVIEW

“Explain Django’s request/response cycle.”

- The WSGI/ASGI server passes the request to Django.
- **Middleware** runs (security, session, auth, CSRF) on the way in.
- The **URL resolver** matches urls.py and calls the **view**.
- The view uses **models** (ORM) and returns a response via a **template** or DRF **serializer**.
- Middleware runs again on the way out; the server sends the response.

> **Follow-up they ask next:** ”What are signals and when would you avoid them?” (decoupled hooks like post_save; avoid them for core logic because they hide side effects; call a service explicitly).

RECAP REMEMBER THESE

Module 25 in 6 lines

1. MVT: urls → view → models → template/serializer; middleware around it.
2. QuerySets are lazy; field lookups like **price__lt**.
3. **select_related** (JOIN) vs **prefetch_related** (IN query) against N+1.
4. DRF: **Serializer** validates, **ViewSet** + **router** = CRUD API.
5. Settings from env; **check --deploy**; **transaction.atomic**, **F()**.
6. Admin, migrations, tests with **APITestCase**.

PRACTICE NOW

Run the bookstore tests (python manage.py test), then add a /api/authors/ endpoint with a test.